Department of Computer Science

Course Code: CIC201

Course Title: Artificial Intelligence

Fall 2025

Lab 12

Neural Networks Contd.

Objective:

The Model Architectures Lab aims to deepen participants' understanding of neuralnetwork components by exploring activation functions and experimenting with model architectures. Through TensorFlow/Keras implementation, participants will delve into the functionality of activation functions such as ReLU and Sigmoid, gaining insights into their impact on model behavior. The lab further encourages hands-on experimentation, allowing participants to modify the model architecture,explore diverse activation functions, and analyze the resulting effects on training performance and classification outcomes..

Name of Student	
Student ID	
Date of Lab Conducted	
Marks Obtained	
Remarks	
Signature	

UIT University
Department of Computer Science
CIC201 – Artificial Intelligence
Spring 2024

1. Objective

The Model Architectures Lab aims to deepen participants' understanding of neuralnetwork components by exploring activation functions and experimenting with model architectures. Through TensorFlow/Keras implementation, participants will delve into the functionality of activation functions such as ReLU and Sigmoid, gaining insights into their impact on model behavior. The lab further encourages hands-on experimentation, allowing participants to modify the model architecture,explore diverse activation functions, and analyze the resulting effects on training performance and classification outcomes..

2. Instructions

You have to perform the following tasks yourselves. Raise your hand if you face any difficulty in understanding and solving these tasks. Plagiarism is an abhorrent practice and you should not engage in it.

3. How to Submit

Submit lab work using TEAMS

Introduction:

Neural networks, inspired by the human brain, have revolutionized machine learning. This lab delves into model architectures, focusing on key concepts such as activation functions and model customization using TensorFlow/Keras. The provided code demonstrates the implementation of a feedforward neural network for classification, with a particular emphasis on understanding activation functions, model training, and evaluation.

1. Activation Functions:

Activation functions play a pivotal role in neural networks, introducing non-linearity and influencing a model's ability to capture complex patterns. This section explores two fundamental activation functions: Rectified Linear Unit (ReLU) and Sigmoid.

- **ReLU (Rectified Linear Unit):**
  - Function: $f(x) = \max(0, x)$
  - Explanation: ReLU replaces negative values with zero, allowing positive values to pass through unchanged. Its simplicity and effectiveness contribute to its widespread use in hidden layers.
- **Sigmoid:**
  - Function: $f(x) = \frac{1}{1+e^{-x}}$
  - Explanation: Sigmoid squashes values to the range [0, 1]. Commonly employed in the output layer for binary classification, it ensures predictions are within a probabilistic range.

2. Model Architecture Implementation:

This section introduces the practical application of neural network concepts using TensorFlow/Keras. The provided code constructs a feedforward neural network for binary classification, emphasizing the following aspects:

- **Dataset Creation:**
  - Synthetic dataset creation using the `` `make_classification` `` function from `` `sklearn.datasets` ``.
- **Data Splitting:**
  - Division of the dataset into training and testing sets using `` `train_test_split` `` from `` `sklearn.model_selection` ``.
- **Neural Network Construction:**
  - Building a simple feedforward neural network using `` `tf.keras.Sequential` ``. The architecture includes a dense layer with ReLU activation and an output layer with a sigmoid activation.
- **Model Compilation:**
  - Compiling the model using binary cross-entropy loss and the Adam optimizer, essential components for binary classification tasks.
- **Model Training:**
  - Training the model on the training data, with performance monitoring through accuracy metrics and validation.
- **Model Evaluation:**
  - Assessing the model's accuracy on the test set and generating a confusion matrix for a detailed performance analysis.

```python
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc



# Create a synthetic dataset (assuming it's classification)
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_classes=2, random_state=42)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build a simple feedforward neural network using TensorFlow/Keras
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile the model with binary cross-entropy loss and Adam optimizer
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model on the training data
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.1)

# Evaluate the model on the test set
y_pred = (model.predict(X_test) > 0.5).astype(int)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy on the test set: {accuracy * 100:.2f}%")

# Create a confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")

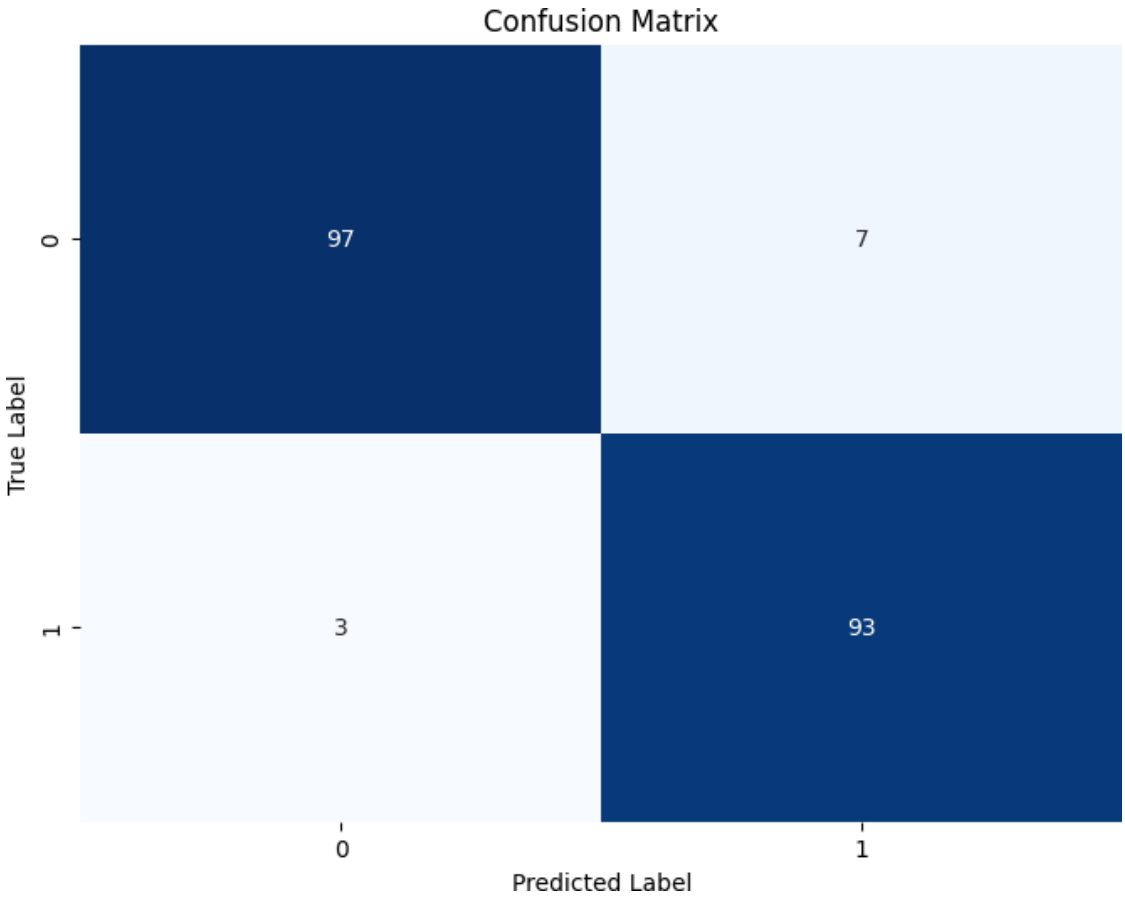
# Visualize the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='g', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

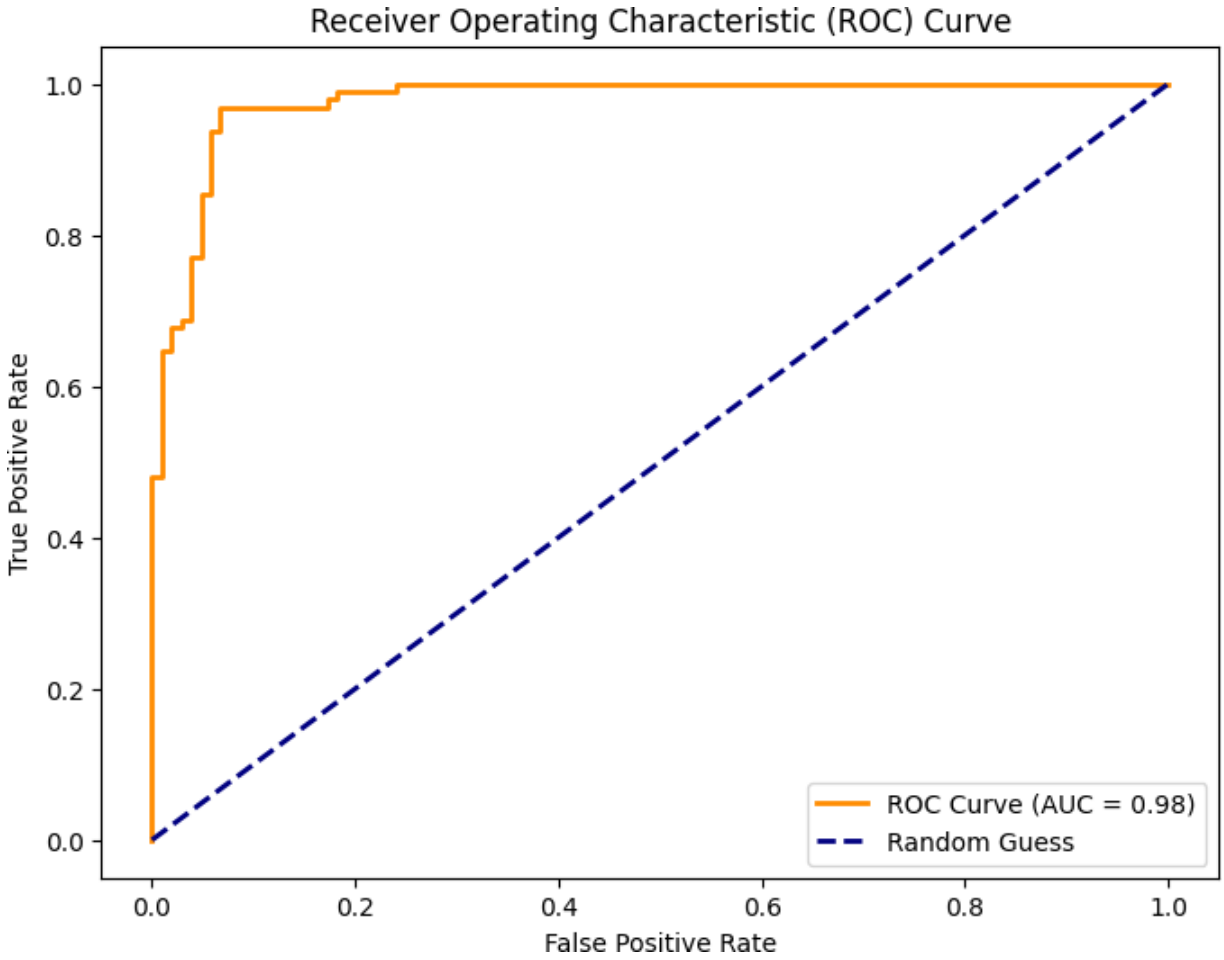
# Compute ROC curve and AUC
fpr, tpr, _ = roc_curve(y_test, model.predict(X_test))
roc_auc = auc(fpr, tpr)

# Visualize the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()
```

Output:

```
Epoch 1/10
23/23 [==============================] - 1s 11ms/step - loss: 0.6386 - accuracy: 0.6403 - val_loss: 0.5382 - val_accuracy: 0.76
25
Epoch 2/10
23/23 [==============================] - 0s 2ms/step - loss: 0.4075 - accuracy: 0.8625 - val_loss: 0.3802 - val_accuracy: 0.875
0
Epoch 3/10
23/23 [==============================] - 0s 2ms/step - loss: 0.3192 - accuracy: 0.8944 - val_loss: 0.3039 - val_accuracy: 0.912
5
Epoch 4/10
23/23 [==============================] - 0s 2ms/step - loss: 0.2728 - accuracy: 0.9028 - val_loss: 0.2601 - val_accuracy: 0.937
5
Epoch 5/10
23/23 [==============================] - 0s 2ms/step - loss: 0.2403 - accuracy: 0.9236 - val_loss: 0.2220 - val_accuracy: 0.962
5
Epoch 6/10
23/23 [==============================] - 0s 2ms/step - loss: 0.2168 - accuracy: 0.9292 - val_loss: 0.2038 - val_accuracy: 0.950
0
Epoch 7/10
23/23 [==============================] - 0s 2ms/step - loss: 0.1990 - accuracy: 0.9361 - val_loss: 0.1830 - val_accuracy: 0.950
0
Epoch 8/10
23/23 [==============================] - 0s 2ms/step - loss: 0.1821 - accuracy: 0.9417 - val_loss: 0.1702 - val_accuracy: 0.950
0
Epoch 9/10
23/23 [==============================] - 0s 2ms/step - loss: 0.1676 - accuracy: 0.9486 - val_loss: 0.1588 - val_accuracy: 0.962
5
Epoch 10/10
23/23 [==============================] - 0s 2ms/step - loss: 0.1559 - accuracy: 0.9486 - val_loss: 0.1429 - val_accuracy: 0.975
0
Accuracy on the test set: 95.00%
Confusion Matrix:
[[97  7]
 [ 3 93]]
```


Confusion Matrix
97
7
3
93
0
1
True Label
0
1
Predicted Label



ReLU and Sigmoid: Activation functions introduce non-linearity. Adam: An optimizer that adjusts weights during training. Binary Cross-Entropy: A loss function for binary classification, measuringthe difference between predicted and true labels.
ReLU (Rectified Linear Unit):

f(x)=max(0,x) - It replaces negative values with zero and allows positive values to pass throughunchanged.
Sigmoid:

- Squashes values to the range [0, 1]. Often used in the output layer for binary classification.

Optimizer (Adam):

An optimizer is an algorithm that adjusts the weights of a neural network during training. It aimsto minimize the loss function. Adam (short for Adaptive Moment Estimation)
Loss Function (Binary Cross-Entropy):

The loss function measures how well the model's predictions match the actual target values. For binary classification, binary cross-entropy is commonly used.

Lab Tasks

Task 1 Deeply study different activation functions also explain their functionality.Modify the previous model to use different activation functions.

Task 2 Experiment with model architecture changes.